

SYLLABUS FOR STATE EXAM – “COMPUTER SCIENCE”

COMPUTER SCIENCE FUNDAMENTALS

1. Sets. Cartesian product. Relations. Functions.

- Axiomatization of sets – axioms of extensionality, separation, power set, and inductively defined sets. Mathematical induction. Basic set operations and their properties.
- Ordered pairs and n-tuples. Cartesian product and generalized Cartesian product of sets.
- Relation over n domains. Properties of binary relations. Equivalence relations and equivalence classes. Partial orders. Hasse diagrams. Total orders. Minimal and maximal elements in partial orders. Embedding a partial order into a total order – topological sorting.
- Partial and total functions. Injections, bijections, and surjections. Definition of finite set and its cardinality. Countably infinite sets. Pigeonhole principle.

2. Basic combinatorial principles and configurations. Recurrence equations.

- Combinatorial principles – pigeonhole, bijection, addition, subtraction, multiplication, division, inclusion-exclusion (with proofs).
- Basic configurations: ordered/unordered, with/without repetition. Derivation of counting formulas. Binomial coefficients and Newton's identities. Combinatorial proofs (double counting).
- Solving linear recurrence relations with constant coefficients – homogeneous and nonhomogeneous cases.

3. Graphs. Trees. Graph traversals.

- Definitions of finite directed (multi)graph and undirected (multi)graph. Paths and cycles. Connectivity and connected components.
- Definition of tree and rooted tree. Proof that any rooted tree is a tree and $|V| = |E| + 1$.
- Spanning tree. Breadth-first and depth-first traversals. Eulerian traversals. Existence theorems for Eulerian cycles and paths (with proofs).

4. Finite automata. Regular languages.

- Deterministic and nondeterministic finite automata. Equivalence of NFA and DFA (with proof).
- Regular operations. Closure properties of regular languages. Regular languages. Kleene's theorem – statement and proof.

- Pumping lemma for regular languages – statement and proof. Examples of non-regular languages. Myhill-Nerode theorem – statement and proof. Minimization algorithm for DFAs.

5. Context-free grammars and languages. Pushdown automata.

- Context-free grammars, parse trees, context-free languages. Closure properties of context-free languages – with proofs.
- Nondeterministic pushdown automata. Execution and language recognition. Equivalence to CFGs – with proof.
- Pumping lemma for context-free languages. Non-context-free languages – examples and proofs. Non-closure under intersection and complement.

6. Algorithm complexity. Asymptotic behavior of integer functions (O-, Ω -, Θ -, o- и ω -notation). Recursive algorithm complexity.

- Models of computation – Turing machine, RAM machine, programming language.
- Time and space complexity – worst and average case. Big-O, Omega, Theta, little-o, and little-omega notations. Properties and limit theorems (without proofs).
- Master theorem for solving:

$$T(1) = \Theta(1), T(n) = a.T\left(\frac{n}{b}\right) + f(n), n > 1$$

7. Graph algorithms with edge weights. Complexity analysis.

- Minimum spanning tree – definition and conditions. Prim's and Kruskal's algorithms.
- Shortest path – Dijkstra's algorithm and for graphs with constant weights. Floyd's algorithm.

8. Dynamic programming. Complexity analysis.

- Essence of dynamic programming. Principle of optimality. Memoization.
- Problems with linear, triangular, and rectangular subproblem tables – LCS, LIS, Knapsack.

COMPUTER SCIENCE CORE

9. Computer architectures. Data formats. Internal structure of the central processor – units and pipeline processing, instructions.

- General structure of computers and conceptual instruction execution, stored program.
- Data formats – integer binary numbers, binary-decimal numbers, floating point binary numbers, signed data and code tables.
- Central processor – registers, ALU, status and flag registers, control units, memory interface, instruction decoding, branching.

10. Memory structure and hierarchy. Segmented and paged addressing. Interrupt system – priorities and handling.

- Structure of main memory. Hierarchy – cache, main and virtual memory.
- Segmented and paged addressing – selector, descriptor, tables and registers in segmented addressing, page directory, descriptor, page replacement strategies in paged addressing.
- Interrupt system – types of interrupts, structure and processing, concurrency and priorities, interrupt controllers.

11. File system. Logical organization and physical representation.

- Logical organization of a file system (FS). File names. File types – regular file, special file, directory, symbolic link, pipe. Internal file structure. File attributes. Hierarchical FS organization – absolute and relative path names, current directory.
- Physical organization of FS. Disk space management strategies. System structures storing disk allocation info: for free blocks, per-file allocated blocks, FS parameters. Examples: UNIX System V, LINUX, MS DOS, NTFS.

12. Process management and inter-process communication.

- Basic system primitives for process management. Process creation. Program execution. Process termination. Synchronization with child process termination. Process rights – user identifiers. Process groups and sessions.
- Mechanisms for inter-process communication. Signals. Pipes. UNIX System V IPC package: Shared memory. Semaphores. Messages.

13. Computer networks and protocols – OSI model. Routing. Protocols IPv4, IPv6, TCP, DNS.

- OSI model – general characteristics of the layers, comparison with the TCP/IP model.
- Distributed routing – distance vector and link-state algorithms.
- IPv4 addressing – classful and classless. Main features of IPv6. TCP – three-way handshake procedure. DNS – basic characteristics (IPv4 and IPv6 name resolution).

14. Procedural programming – basic constructs.

- Principles of structured programming.
- Control of computation. Main control structures – conditionals, loops.
- Variables – types: local and global, initialization, assignment.
- Functions and procedures. Parameters – types and passing (by name, by value). Types and type checking.
- Strings – representation in memory and basic operations.

15. Procedural programming – pointers, arrays, recursion.

- Pointers and pointer arithmetic.
- One-dimensional and multi-dimensional arrays. Basic operations with arrays – indexing. Sorting and searching in a one-dimensional array – basic algorithms.
- Recursion – direct and indirect recursion, linear and branched recursion.

16. Object-oriented programming. Core principles. Classes and objects. Inheritance and encapsulation.

- Abstraction with data structures. Classes and objects. Class declaration and object declaration. Main types of constructors. Resource and dynamic memory management (RAII). Methods – declaration, parameter passing, return values.
- Inheritance. Derived and nested classes. Access to inherited components. Encapsulation and information hiding. Static fields and methods.

17. Object-oriented programming. Subtype and parametric polymorphism. Multiple inheritance.

- Virtual functions and subtype polymorphism. Dynamic binding. Abstract methods and classes. Arrays of objects and arrays of pointers to objects.
- Parametric polymorphism. Function and class templates.
- Multiple inheritance.

18. Data structures. Stack, queue, list, tree. Basic operations and implementation.

- Data structures – definition of the concept.
- List. Logical description. Singly and doubly linked lists. Implementation characteristics. Complexity of add, remove, search. Class definition for a list using one implementation.
- Stack. Logical description. Static, dynamic, and linked implementation. Complexity of add and remove. Class definition using one implementation.
- Queue. Logical description. Static, dynamic, and linked implementation. Complexity of add and remove. Class definition using one implementation.
- Tree data structures – general tree and binary tree. Logical description. Memory representation. Class definition for a tree.
- Binary search tree. Logical description. Memory representation. Complexity of add, remove, search. Class definition for BST.

19. Functional programming. General features of the functional style. Function definition and use. Evaluation models. Higher-order functions.

- Functional style features. Main components of functional programs. Primitive expressions. Means of combination and abstraction. Expression evaluation. Function definition and application. Evaluation models – applicative (strict, call-by-value) and normal (lazy, call-by-name).
- Higher-order functions. Functions as parameters and return values. Anonymous (lambda) functions.

20. Functional programming. Lists. Streams and lazy evaluation.

- Lists. Representation. Basic operations. Higher-order functions for list processing.
- Infinite streams and lists. Basic operations and higher-order functions. Lazy evaluation. Working with infinite streams.

21. Syntax and semantics of terms and formulas in First-order predicate logic. Unification.

- Definition of the syntactic notions: term and formula in a given first-order logic language. Definition of unifier and most general unifier (MGU) for a set of terms. Presentation of the unification algorithm for computing an MGU for a finite set of terms.
- Definition of semantic interpretation of terms and formulas in a given structure for the language. Proof: a set of closed universal formulas has a model if and only if the set of its ground instances is propositionally satisfiable.

22. Resolution method in propositional and First-order predicate logic. Horn Clauses.

- Definition of propositional resolution derivation. Statement and proof of the soundness and completeness theorem for resolution. Description of the resolution method in first-order logic.
- Definition of Horn clause. Proof: every satisfiable set of Horn clauses has a least (minimal) model.

23. Databases. Relational data model.

- Relational data model: domain, relation, tuple, attribute, relation schema, relational database schema, database implementation, operations on relational databases, queries.
- Relational algebra: basic operations (union, difference, Cartesian product, projection, selection) and additional operations (intersection, division, join, natural join).

24. Databases. Normal forms.

- Normal forms. Relational database schema design. Anomalies, constraints, keys.
- Functional dependencies, Armstrong's axioms. First, second, third normal form, Boyce-Codd normal form.
- Multivalued dependencies, axioms of functional and multivalued dependencies, lossless join, fourth normal form.

25. State space search. Genetic algorithms.

- State space. Basic concepts.

- Formulation of main types of search problems in state space: pathfinding to a goal, game strategy for two players, constraint satisfaction problems.
- Heuristic (informed) search methods: Best-first search, Beam search, Hill climbing, A*.
- Genetic algorithms – core algorithm, types of crossover and mutation.

26. Modern software technologies.

- Software product and process. Software process models. Software technologies. Software project and resource management. Participants in the software process.
- Functional and non-functional requirements. Analysis and design of software requirements. Software design. Object-oriented design. Description languages. UML.
- Software verification and validation. Software testing. Test process management. Quality management of the software development process.
- Modern software technologies. Agile methodologies. Extreme Programming (XP). Test Driven Development. Feature Driven Development. SCRUM.

27. Software system architectures.

- Software technologies centered on software architecture. Quality attributes – architecture design. Design for performance, complexity, scalability, heterogeneity, and adaptability. Reliability and security.
- Components and connectors. Types of connectors and their variable characteristics. Criteria for selecting appropriate connectors.
- Architectural styles. Distributed, network-based, decentralized architectures. Service-oriented architectures and web services. Client-server. Software architecture analysis and visualization.

MATHEMATICS AND APPLICATIONS

28. Symmetric operators in finite-dimensional Euclidean spaces. Basic properties. Diagonalization theorem.

- Definition of symmetric operator.
- Matrix of a symmetric operator in orthonormal basis. All eigenvalues of symmetric operators are real. Eigenvectors corresponding to distinct eigenvalues are orthogonal.
- There exists an orthonormal basis in which the matrix is diagonal.

29. Symmetric and alternating groups. Cayley's theorem. Group homomorphism theorem.

- Symmetric group S_n – representation of elements as product of disjoint cycles. Conjugation of elements in S_n . Transpositions - representation of elements as products of transpositions. Alternating group.

- Cayley's theorem – every finite group is isomorphic to a subgroup of the symmetric group.
- Group homomorphism, kernel and image. Group homomorphism theorem.

30. Fermat's theorem. Mean value theorems (Rolle's, Lagrange's, and Cauchy's). Taylor's formula.

- Define the concept of local extremum of a function of one variable. Formulate and prove a necessary condition for local extremum of differentiable functions (Fermat's theorem).

Let the function f be continuous on the closed interval $[a, b]$ and possess a derivative at least on the open interval (a, b) . To be proved that:

- Rolle's Theorem $\rightarrow$ if $f(a) = f(b)$, then there exists $c \in (a, b)$ such that $f'(c) = 0$ (The Weierstrass Theorem can be used without proof – every continuous function on a finite closed interval reaches its maximum and minimum.)
- Lagrange's Theorem $\rightarrow$ there exists $c \in (a, b)$ such that $f(b) - f(a) = f'(c)(b - a)$
- Cauchy's Theorem $\rightarrow$ if the function g is continuous on the closed interval $[a, b]$ and has a derivative at least on the open interval (a, b) , and $g'(x) \neq 0$ for $x \in (a, b)$, then there exists $c \in (a, b)$ such that $\frac{f(b)-f(a)}{g(b)-g(a)} = \frac{f'(c)}{g'(c)}$. Prove that under the stated assumptions: $g(a) \neq g(b)$
- Derive Taylor's formula with remainder term in Lagrange form.

31. Definite integral. Definition and properties. Integrability of continuous functions. Newton–Leibniz theorem.

- Define partition of an interval, upper and lower Darboux sums. Establish that when adding new points to the partition of the interval, the upper Darboux sums do not increase, and the lower Darboux sums do not decrease (with a diagram). Define the Riemann integral using the Darboux approach.
- Prove that a given function is Riemann integrable if and only if for every $\varepsilon > 0$ there exist an upper Darboux sum S and a lower Darboux sum s such that $S - s < \varepsilon$. Using this theorem and Cantor's Theorem (every continuous function on a finite closed interval is uniformly continuous), prove that every continuous function on a finite closed interval is Riemann integrable.
- List (without proof) the main properties of the Riemann integral.

Prove that if f is continuous on $[a, b]$ then there exists $c \in [a, b]$ such that:

$$\int_a^b f(x)dx = f(c)(b - a)$$

For this statement apply (without proof) the property of integrating inequalities and the theorem that every continuous function on $[a, b]$ takes all values between its maximum and minimum.

- Prove the Newton-Leibniz: if f is continuous on $[a, b]$, then for every $x \in [a, b]$:

$$\frac{d}{dx} \int_a^x f(t) dt = f(x)$$

Show how the theorem is used to compute definite integrals.

32. Equations of a line and a plane. Distance formulas.

- Vector and parametric (scalar) equations of line and plane.
- General equation of a line in the plane. Cartesian equation. Relative position of two lines. Normal form of a line. Distance from a point to a line.
- General equation of a plane. Relative position of two planes. Normal form of a plane. Distance from a point to a plane.

33. Iterative methods for solving nonlinear equations.

- Define the concept of fixed point of a mapping and prove that if φ is a continuous mapping from $[a, b]$ into itself, then φ has at least one fixed point in $[a, b]$. Show that solving the equation $f(x) = 0$ can be reduced to finding a fixed point.
- Define the concept of contraction mapping and prove that if φ is continuous and contracting with Lipschitz constant $q < 1$ on $[a, b]$, then:
 - the equation $x = \varphi(x)$ has a unique root ξ in $[a, b]$;
 - the sequence $\{x_n\}$ of successive approximations (with arbitrary $x_0 \in [a, b]$ and $x_{n+1} = \varphi(x_n)$, $n = 0, 1, 2, \dots$) converges to ξ as $n \rightarrow \infty$, such that $|x_n - \xi| \leq (b - a)q^n$, $\forall n$
- To obtain as a consequence that if ξ is a root of the equation $x = \varphi(x)$ and φ has a continuous derivative in a neighborhood U of ξ , for which $|\varphi'(\xi)| < 1$, then for a sufficiently good initial approximation x_0 the iterative process generated by φ converges with the rate of a geometric progression.
- Define the concept of order of convergence. Provide a geometric illustration, the formula for successive approximations, and the order of convergence for: the method of chords, the secant method, and Newton's. Prove that in the method of chords, the convergence is with the rate of a geometric progression (under the condition that the root is isolated in a sufficiently small interval).

34. Discrete distributions. Uniform, binomial, geometric, and Poisson distributions. Example problems. Moments – expectation and variance.

- Definition of discrete probability distribution of a random variable. Properties of probabilities: non-negativity, normalization, monotonicity and additivity.
- For each of the distributions, provide an example in which it arises. Calculate the mathematical expectation and variance of each of these distributions. In the calculations, a generating function or moment-generating function may be used, but it must be defined, and its main properties must be shown (without proof).